Checkpoint 1

Christopher Samuel Fong 方文豪
112006233

ThreadCreate:

```
chris@Chris MINGW64 /d/NTHU FILES/SEMESTER 6/OS/ppc1
$ make clean
rm: cannot remove '*.lnk': No such file or directory
Makefile:25: recipe for target 'clean' failed
make: *** [clean] Error 1

chris@Chris MINGW64 /d/NTHU FILES/SEMESTER 6/OS/ppc1
$ make
sdcc -c  testcoop.c
sdcc -c  cooperative.c
cooperative.c:26:1: warning: multi-line comment
   26 | // #define SAVESTATE                 \
      | ^
cooperative.c:42:1: warning: multi-line comment
   42 | // #define RESTORESTATE              \
      | ^
cooperative.c:274: warning 85: in function ThreadCreate unreferenced function argument : 'fp'
sdcc  -o testcoop.hex testcoop.rel cooperative.rel
```

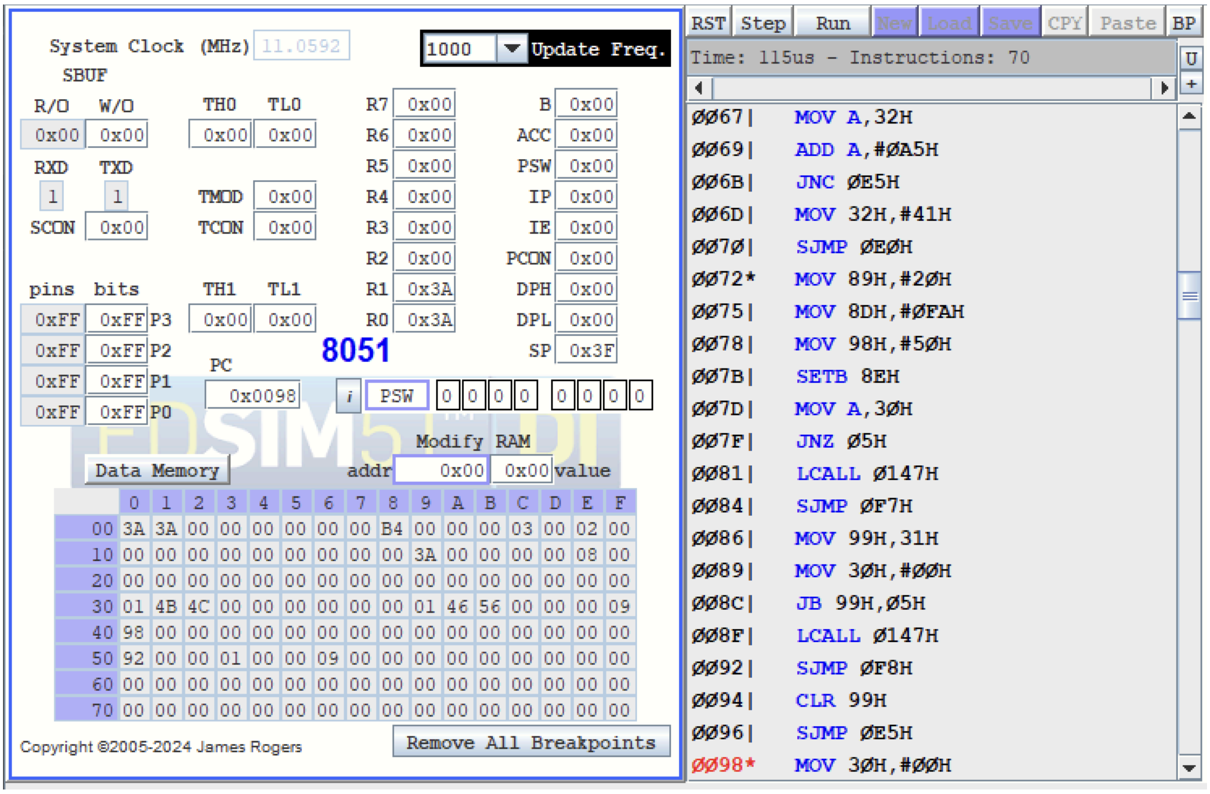
Functions:

```
Area                                    Addr        Size        Decimal Bytes (Attributes)
--------------------------------        ----        ----        ------- ----- ------------
CSEG                                0000004F    0000019C =         412. bytes (REL,CON,CODE)

      Value  Global                              Global Defined In Module
      -----  --------------------------------   ------------------------
C:   0000004F  _Producer                          testcoop
C:   00000072  _Consumer                          testcoop
C:   00000098  _main                              testcoop
C:   000000A4  __sdcc_gsinit_startup              testcoop
C:   000000A8  __mcs51_genRAMCLEAR                testcoop
C:   000000A9  __mcs51_genXINIT                   testcoop
C:   000000AA  __mcs51_genXRAMCLEAR               testcoop
C:   000000AB  _Bootstrap                         cooperative
C:   000000C9  _ThreadCreate                      cooperative
C:   00000147  _ThreadYield                       cooperative
C:   00000199  _ThreadExit                        cooperative
```

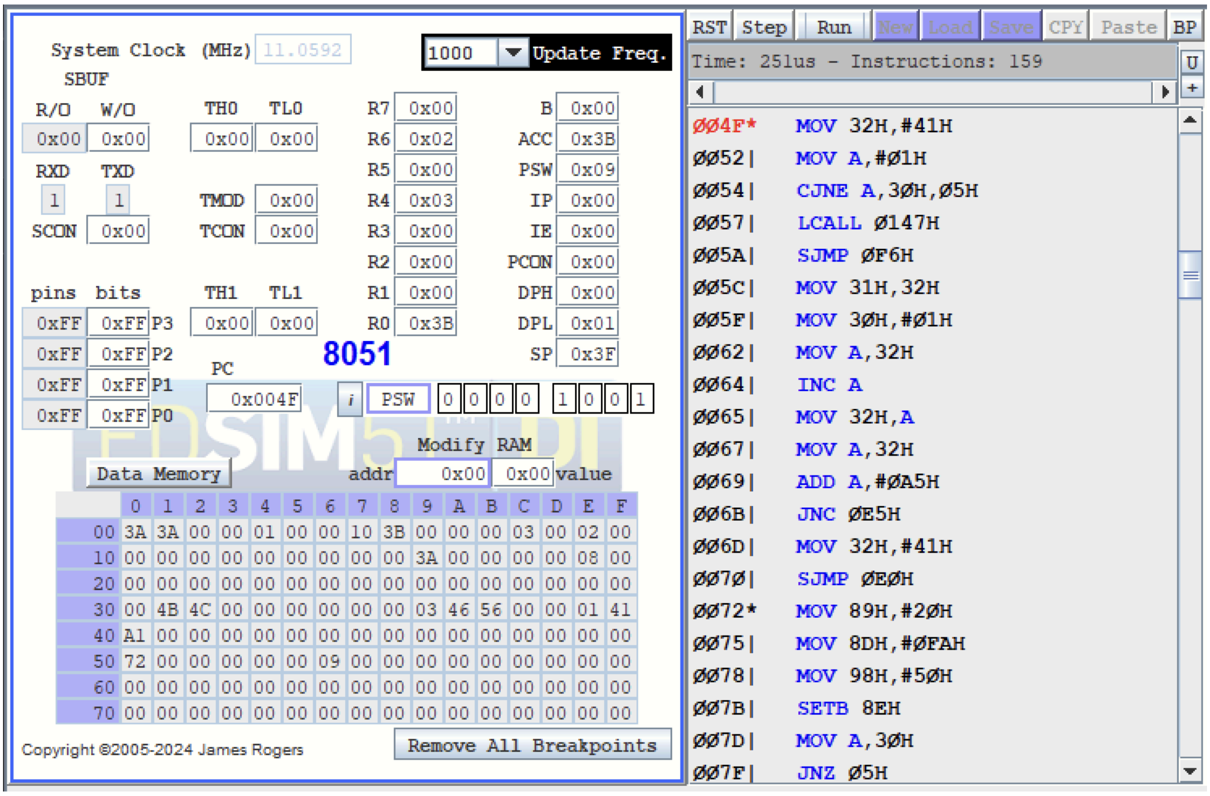
Variables:

```
Area                               Addr        Size        Decimal Bytes (Attributes)
--------------------------------   ----        ----        ------- ----- ------------
.  .ABS.                       00000000    00000000 =          0. bytes (ABS,CON)

      Value  Global                              Global Defined In Module
      -----  --------------------------------   ------------------------
     00000000  .__.ABS.                           cooperative
     00000030  _data_ready                        testcoop
     00000031  _shared_buf                        testcoop
     00000032  _curr_char                         testcoop
     00000038  _active_thread                     cooperative
     00000039  _thread_mask                       cooperative
     0000003A  _saved_sp                          cooperative
     0000003E  _temp_tid                          cooperative
     0000003F  _sp_temp                           cooperative
     00000080  _P0                                cooperative
     00000080  _P0_0                              cooperative
     00000081  _P0_1                              cooperative
     00000081  _SP                                cooperative
     00000082  _DPL                               cooperative
     00000082  _P0_2                              cooperative
     00000083  _DPH                               cooperative
     00000083  _P0_3                              cooperative
     00000084  _P0_4                              cooperative
     00000085  _P0_5                              cooperative
     00000086  _P0_6                              cooperative
     00000087  _P0_7                              cooperative
```

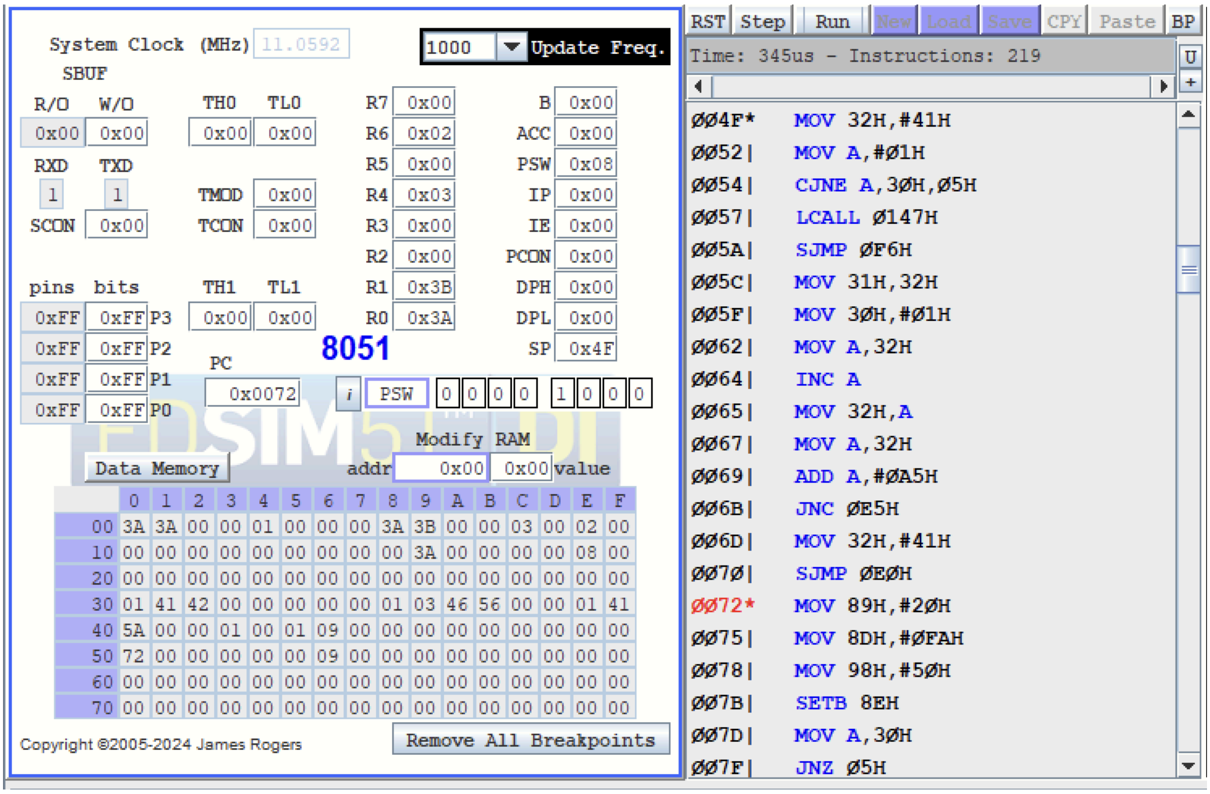
Main:



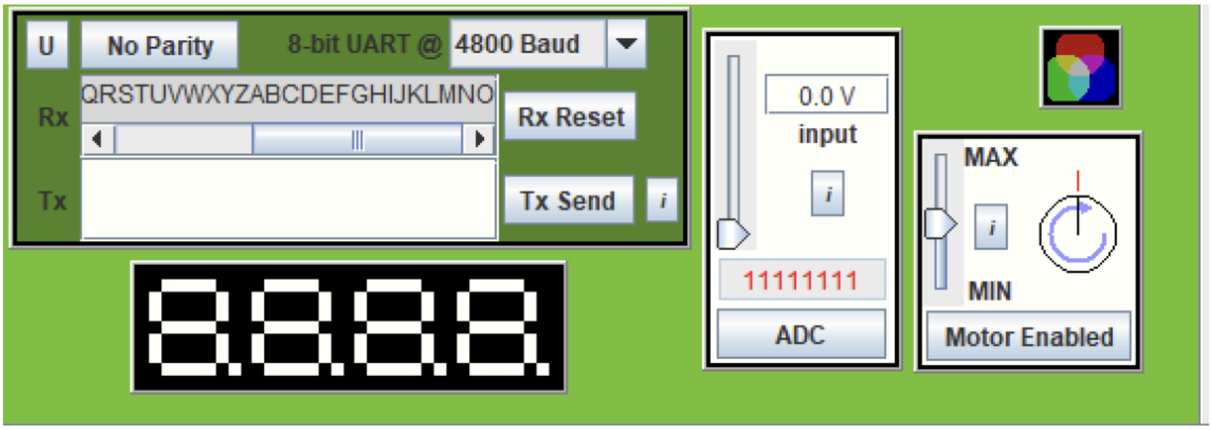
Producer:



Consumer:



Output:



Explanation:

Implementation Overview

To implement the producer-consumer model, I allocated shared variables in the 0x30 memory range. The Producer runs inside the main thread (Thread 0), continually generating characters and yielding when the buffer is full. The Consumer is dynamically spawned as Thread 1, fetching data, sending it to the serial port, and yielding while waiting for the transmission to finish. I used a bitmap to track active threads and allocated exactly 16 bytes (0x10) for each thread's stack, with the base starting at 0x3F.

Before ThreadCreate

I referenced the map file and set a breakpoint at 007EH, right before the LCALL to the ThreadCreate function. As shown in the screenshot, the stack pointer (SP) is currently at 46H. WHen the call occurs the LCALL pushes a 2-byte return address, incrementing the stack. Once inside the ThreadCreate, the OS calculates the new thread's dedicated stack space using the formula 0x3F + (ThreadID*16). For Thread 1, it temporarily shifts the SP to 4FH to safely push the initial thread context (cleared registers and PSW) without corrupting Thread 0's memory.

How I know the Producer is running

The simulator paused at the address 0073H, which matches the _Producer function in my map file. I can confirm the Producer is running by observing the PSW register, which reads 00H. This means the register bank selection bits (RS1 and RS0) are 00, verifying that the CPU is utilizing Register Bank 0. THis perfectly aligns with my design, where the PRoducer runs inside the main thread (Thread 0). Additionally, the SP is at 3FH, matching Thread 0's stack.

How I know the Consumer is running

Execution stopped at 00A5H, which corresponds to the _Consumer function. I know the Consumer is actively running because a context switch has clearly occurred: the PSW register now reads 08H. The RS0 bit has flipped to 1, confirming the microcontroller is now using Register Bank 1 (which was assigned to Thread 1). Furthermore, the SP is now at 4FH, providing execution is safely isolated within the specific 16-byte memory boundary allocated for the Consumer thread.